

增強課程 7：Explicit Enhancement Point/Section

Lecture

前面幾題的 Enhancement Spot 都是拿來裝 BAdI Definition (en05/en06) 或 Source Code Plugin 的隱式插入點 (en04)。這題要講的是在自己的 Z 程式裡，主動宣告一個「這裡可以被別人插入程式碼」的位置——這是 Explicit (顯式) Enhancement Point，跟 en04 的 Implicit (隱式) 插入點是同一個 Source Code Enhancement 家族底下的兩種宣告方式。

Explicit 與 Implicit 的核心差異：

	Implicit (en04)	Explicit (本題)
插入點怎麼來	框架保證到處都有，不需要原開發者宣告	原開發者要主動在原始碼裡寫 <code>ENHANCEMENT-POINT</code> / <code>ENHANCEMENT-SECTION</code> 語句
找插入點的方式	SE37/SE38 切換「Show Implicit Enhancement Options」才看得到	直接寫在原始碼裡，肉眼就看得得到
能做什麼	只能插入程式碼，不能改變介面	<code>ENHANCEMENT-POINT</code> 只能插入； <code>ENHANCEMENT-SECTION</code> 可以整段取代原有邏輯

兩種語句的語法 (官方文件 `ABAPENHANCEMENT-POINT` / `ABAPENHANCEMENT-SECTION`)：

```
ENHANCEMENT-POINT <name> SPOTS <spot1> [<spot2>] [STATIC].

ENHANCEMENT-SECTION <name> SPOTS <spot1> [<spot2>] [STATIC]
... 原有邏輯 ...
END-ENHANCEMENT-SECTION.
```

`SPOTS` 後面接的 Enhancement Spot，跟 en05/en06 用過的是同一種物件 (`ENHS/XS`)，只是這裡不掛 BAdI Definition，掛的是「Enhancement Spot Element Definition」(技術上叫 Hook Definition，`HOOK_DEF`)。`STATIC` 是給資料宣告用的 (例如額外的 `DATA` 宣告)，不加則是給可執程式碼用的。

⚠️ 「Include Bound」不是決定「追加 vs 整段取代」的開關 (2026-07-30 更正)：Create Option 對話框裡確實有一個「Include Bound」核取方塊 (在 Enhanceable Object 區塊)，但當 **Enhanceable Object Type 是 Program** 時，這個欄位整個是灰階、無法勾選——這代表它在「一般 Z 程式裡宣告 Explicit Enhancement」這個情境下根本用不到，不是「留給你決定行為」的選項。`ENHANCEMENT-SECTION` 的 Implementation 到底是「保留原邏輯疊加」還是「整段取代」，實際行為要靠直接建一個 Implementation、觀察執行結果來確認，不能靠這個灰階欄位判斷。

⚠️ 這題全程都是 GUI-only，而且比 en04/en05/en06 更徹底——連 ADT 讀寫都不支援：

- 建立 **Enhancement Spot** 本身：跟 en05 一樣沒有 ADT 建立 API，但這裡不能像 en05 那樣繞去 SE18——只有 **BAdI Definition** 類型的 **Spot** 能用 SE18 建立，**Source Code Plug-In** 類型必須從 SE38 GUI 建立 (2026-07-30 實測確認)。這不是「SE18 剛好沒開放這個選項」的表面現象，而是兩種類型本質不同：BAdI Definition 是獨立物件，不依附任何程式的特定位置，SE18 這種脫離程式上下文的獨立管理畫

面可以完整建立它；但 Source Code Plug-In 型的 Spot ([ENHANCEMENT-POINT/ENHANCEMENT-SECTION](#) 用的那種) 本質上需要把「這個 Spot 錨定在哪支程式的哪個位置」這個資訊記錄下來，這個錨點只有在該程式的編輯器 (SE38) 裡、對著實際原始碼位置操作才能被正確捕捉——SE18 沒有程式上下文，天生就沒有這個資訊來源，所以唯一能建立這種 Spot 的方式是下面講的「從 SE38 Create Option 對話框裡順便建立」。

- 建立 **Explicit Enhancement Point/Section** 的宣告本身 (**Create Option**)：也是 GUI-only，且這一步驟做完之後，這支程式就再也不能用 **sap_lock/ADT** 編輯了——實測對已經有 Explicit Enhancement 的程式呼叫 **sap_lock**，直接回 **ExceptionResourceIsEnhanced: The editor does not support enhanced objects (use SAP GUI instead)**。這是這門課至今遇過最徹底的 ADT 限制：**en04 的 Source Code Plugin**、**en05/en06 的 BAdI Implementation**，空殼建好之後內容都還能用 ADT 讀寫；這題連讀都讀不到 (**GET Enhancement Spot** 回 **Enhancement technology HOOK_DEF is not supported yet**，**GET Enhancement Implementation** 的原始碼回 **uriMappingError**)。
- 寫實際程式碼 (**Create Implementation**)：一樣 GUI-only，且要直接在 SE38 編輯器裡手動輸入程式碼，Claude 完全幫不上忙，只能提供程式碼文字讓使用者貼上去。

建立流程完整版 ([ENHANCEMENT-POINT](#) / [ENHANCEMENT-SECTION](#) 兩種都適用，2026-07-30 端對端驗證成功)：

1. **SE38** 開啟目標程式 → **Change** (一般編輯模式，不需要切到 Enhance 模式——Create Option 這個動作本身在一般模式的右鍵選單就找得到；下面會另外說明 Enhance 模式真正該在哪個步驟用)
2. 游標點在想插入 Point 的那一行，或選取想包成 Section 的那段程式碼區塊
3. 右鍵 → Enhancement Operations → **Create Option**
4. 彈出的 Create Enhancement Option 對話框：Type and Name 選 **Enhancement Point** 或 **Enhancement Section**，填 Option 名稱；Binding in Source Code 選「as conditional call」(可執行程式碼用這個，不要選 STATIC)；Enhancement Spot 那格直接打一個全新、從未用過的名稱 (⚠ 不要填 SE18 建立的既有 Spot——型別對不上會導致一系列誤導性錯誤，見下方「排錯記錄」)，按 Enter——系統偵測到不存在會詢問是否建立，確認建立即可當場建好一個正確綁定到這支程式的 **Enhancement Spot**
5. 存檔、**Activate**——**Inactive Objects** 清單會同時列出新建的 **ENHS** (Enhancement Spot) 跟 **PROG/REPS** (程式本身)，一次批次啟用即可；原始碼裡的宣告那行會自動改成正確的 **SPOTS** <剛建立的新Spot名稱>
6. 驗證 **Binding**：① SE18 Display 這個新建的 Spot，Attributes 頁籤的 Enhancement Method 應該正確顯示「**Source Code Plug-In**」(不是 BAdI Definition)，Technical Details 頁籤的 Referenced Objects 應該列出這支 Program；② 回到 SE38 編輯器，這個區塊左側裝訂線會多出一個螺旋狀小圖示，這是最快速的目測確認方式——沒有這個圖示代表 Binding 沒成功
7. 看到螺旋圖示、確認 **Binding** 成功之後，按工具列的「**Enhance**」切換按鈕，畫面標題會變成「Change Enhancements for ...」——這一步才是 **Enhance** 按鈕真正該用的時機，不是建立宣告的時候
8. 在切換後的模式下，游標點在 **SPOTS** 那一行 → 右鍵 → Enhancement Operations → 這時候 **Create Implementation** 才會是可用選項 (沒切換模式、或 Binding 還沒成功時這個選項不會正確作用)；點下去填 Implementation 名稱、套件，存檔 Activate，會產生一個可編輯的 **ENHANCEMENT n** <名稱>，... **ENDENHANCEMENT** 區塊
9. 在這個區塊裡直接手動輸入要插入的程式碼、存檔、Activate

「**Enhance**」切換按鈕的正確角色 (已完整驗證)：不是「建立宣告 (Create Option) 之前要先按」，而是「確認 **Binding** 成功 (看到螺旋圖示) 之後、要開始編輯 **Enhancement** 內容 (**Create/Change Implementation**) 之前才按」——上面第 1~6 步 (Create Option) 全程用一般 Change 模式；第 7~9 步 (Create Implementation) 才需要切到 Enhance 模式。兩個階段的操作模式不同、時機也不同，不要混為一談。

排錯記錄 (2026-07-29 走過的彎路，供對照)：第一輪嘗試時，先用 SE18 獨立建立了一個 Enhancement Spot (結果型別是 BAdI Definition)，再拿去套用到 ENHANCEMENT-SECTION，結果得到一系列文字通順但語意誤導的錯誤訊息，讓人誤以為是物件設定/命名/綁定的問題：

- ED291 Creating nested enhancements is not supported (長文聲稱游標在 SECTION/END-SECTION 中間，但實際游標可能根本不在那個位置)
- Enhancement spot <name> defines enhancement spot in another object (聲稱 Spot 綁定到別的物件，實際上是型別不合，換任何 Spot 名稱重試都一樣會報錯)

當時一度誤判「要先按 Enhance 切換按鈕」才是解法，2026-07-30 重新端對端測試後確認這個歸因不準確：真正根因是 SE18 建的 Spot 型別 (BAdI Definition) 從一開始就不適用於 Explicit Point/Section，換成「直接在 Create Option 對話框裡打新名稱建立」(上面第 4 步) 就完全不需要碰 Enhance 按鈕，一次成功。這個排錯過程本身是很好的教材：遇到語意通順但邏輯對不上的錯誤訊息時，值得懷疑是不是「用錯了物件類型」而不是「操作步驟少做了什麼」。

實測結果 (ZR_EN07_EXPLICIT_DEMO)：

```
lv_text = |Standard: processing carrier { lv_carrid }|.
WRITE: / lv_text.

ENHANCEMENT-POINT ep_en07_after_init SPOTS zes_en07_v3.
* 插入的程式碼：
*   lv_text = |Enhanced: extra greeting inserted via ENHANCEMENT-POINT for carrier {
lv_carrid }|.
*   WRITE: / lv_text.

WRITE: / lv_text.
```

執行輸出：

```
Standard: processing carrier LH
Enhanced: extra greeting inserted via ENHANCEMENT-POINT for carrier LH
Enhanced: extra greeting inserted via ENHANCEMENT-POINT for carrier LH
```

第二行是插入點本身新增的輸出；第三行是插入點之後、原本就存在的 **WRITE: / lv_text.**——因為插入的程式碼改了共用變數 **lv_text**，連帶影響了它後面既有的邏輯，證實 Explicit Enhancement Point 插入的程式碼是跟原有程式碼在**同一個變數作用域**裡執行的，不是隔離的沙箱。

ENHANCEMENT-SECTION：跟 **ENHANCEMENT-POINT** 語法幾乎一樣，差別是 **ENHANCEMENT-SECTION ... END-ENHANCEMENT-SECTION** 包住一段**既有邏輯**。官方文件籠統描述 Enhancement Implementation 可以「保留原邏輯、額外追加」或「整段取代原邏輯」，但**這套系統實測結果是：只要建立了 Implementation，就是整段取代，沒有追加這條路**（詳見下方實測結果）——原因是 Create Option 對話框裡的「Include Bound」欄位在 Enhanceable Object 是 Program 型別時整個灰階、無法勾選（見上方「Lecture」段落的更正說明），這個系統沒有開放讓你選「追加」的入口。

實測結果 (ZR_EN07_SECTION_DEMO，2026-07-30)：依照上面的「建立流程完整版」，選取一段預設邏輯（計算並顯示一筆預設金額）當作 Section 範圍，Create Option 建立 **ENHANCEMENT-SECTION es_mytest**

SPOTS zes_en07_section_v1 · Spot 自動建立並正確 Binding (SE18 確認 Enhancement Method = Source Code Plug-In · 編輯器左側裝訂線出現螺旋圖示) :

```
REPORT zr_en07_section_demo.

DATA: lv_carrid TYPE s_carr_id VALUE 'LH',
      lv_text    TYPE string.

WRITE: / 'EN07 Explicit Enhancement Section demo'.
WRITE: / '====='.

ENHANCEMENT-SECTION es_mytest SPOTS zes_en07_section_v1.
" -----
" 系統預設會執行的程式碼 (Default Logic)
" 若未來沒有建立 Implementation · 系統就會執行這一段
" -----
DATA: lv_amount TYPE i.
lv_amount = 100.
WRITE: / '預設金額:', lv_amount.
END-ENHANCEMENT-SECTION.

*ENHANCEMENT-SECTION es_en07_greeting SPOTS zes_en07_section_v1.
*lv_text = |Standard: greeting for { lv_carrid }|.
*END-ENHANCEMENT-SECTION.

WRITE: / lv_text.
```

****倒數第二段的 *ENHANCEMENT-SECTION es_en07_greeting ... 是排錯過程留下的殘留片段，已被整段註解掉、不會執行****：這是排錯初期第一次嘗試建立 Section 時取的 Option 名稱 (es_en07_greeting · 呼應 ENHANCEMENT-POINT 案例的 lv_text = |Standard: greeting for ...| 寫法) · 後來改用 ES_MYTEST + 預設金額邏輯重新成功建立 · 這段舊嘗試就留在原始碼裡當註解、沒有刪除。這也解釋了最後一行 WRITE: / lv_text. 為什麼會印出空字串——lv_text 從頭到尾沒有被賦值過 (賦值那行在註解裡) · 這是真實系統目前的原樣，不是示範邏輯的一部分。

Create Implementation 實測 (ZEI_EN07_SECTION_APPEND · 2026-07-30)：先踩到一個很有意義的語法錯誤——Implementation 程式碼直接沿用原邏輯裡宣告的 lv_amount 會報 Field "LV_AMOUNT" is unknown · 代表 Implementation 的作用域跟原邏輯區塊是分開的，原邏輯裡的區域宣告對 Implementation 不可見。改用 Implementation 自己獨立宣告的變數後正常啟用：

```
ENHANCEMENT 1 ZEI_EN07_SECTION_APPEND.
WRITE: / '=== Implementation 有執行到這裡 ==='.
DATA: lv_new_amount TYPE i.
lv_new_amount = 999.
WRITE: / 'Implementation 設定的金額:', lv_new_amount.
ENDENHANCEMENT.
```

執行輸出：

```
EN07 Explicit Enhancement Section demo
=====
=== Implementation 有執行到這裡 ===
Implementation 設定的金額：          999
```

「預設金額: 100」完全沒有出現——證實這套系統裡 **ENHANCEMENT-SECTION** 一旦建立 Implementation，就是整段取代：原邏輯（含 **DATA: lv_amount** 那行宣告、**lv_amount = 100**、**WRITE '預設金額:'**）完全不執行，Implementation 提供的程式碼是唯一生效的邏輯。結合前面 Include Bound 灰階不可用的觀察，可以下一個明確結論：在這套系統對一般 Z 程式（**Program** 型別）建立 **Explicit ENHANCEMENT-SECTION** 時，沒有「保留原邏輯、額外追加」這個選項可用，只有整段取代——官方文件描述的「追加」能力，理論上需要 Include Bound 開放的情境（例如某些 BAdI/框架物件），不適用於這裡的純 Program 場景。

學習目標

- 能講出 Explicit 與 Implicit Enhancement Point 的差異：前者需要原開發者主動宣告、肉眼可見；後者框架保證到處都有、需要切換顯示才看得到
- 能講出 **ENHANCEMENT-POINT**（純插入）與 **ENHANCEMENT-SECTION**（可整段取代）的能力差異，並知道在這套系統的 **Program** 型別情境下，**ENHANCEMENT-SECTION** 實測只有「整段取代」，沒有「追加」的選項可用（Include Bound 灰階不可勾選）
- 能解釋為什麼 Implementation 程式碼引用原邏輯裡宣告的區域變數會報 **Field ... is unknown**：Implementation 的作用域跟被取代的原邏輯是分開的，原邏輯裡的 **DATA** 宣告對 Implementation 不可見，Implementation 要自己獨立宣告變數
- 知道 Explicit Enhancement 的建立與內容編輯完全是 **GUI-only**，而且一旦程式碼裡有了 Explicit Enhancement，這支程式就再也不能用 ADT **sap_lock** 編輯（**ExceptionResourceIsEnhanced**）——這是本課程遇過最徹底的 ADT 限制
- 知道 **SE18** 建立 **Enhancement Spot** 只能選 **BAdI Definition** 類型，沒有 **Source Code Plug-In** 選項——Explicit Point/Section 用的 Spot 必須從 SE38 的 Create Option 對話框「順便」建立，不能靠 SE18 事先準備
- 知道 Create Option（建立 Point/Section 宣告 + Spot）這一步不需要切到 Enhance 模式，一般 Change 模式的右鍵選單就找得到；Enhance 模式是用在編輯「已存在的 Enhancement 內容」（Create/Change Implementation）這個不同的步驟
- 能講出「遇到語意通順但邏輯對不上的錯誤訊息（如 nested enhancements、spot defines enhancement spot in another object）時，優先懷疑用錯物件類型，而不是操作步驟少做了什麼」這個排錯心法
- 能用「編輯器左側裝訂線是否出現螺旋圖示」快速目測 Enhancement Spot 是否已正確 Binding 到程式，不用每次都跑去 SE18 查 Technical Details
- 能解釋為什麼插入的程式碼會影響插入點之後的既有邏輯（同一個變數作用域，不是隔離環境）

事前準備（已於本系統 client 130 實際完成，非假設）

1. **ZES_EN07_V3**（\$TMP，使用者於 **SE38 Create Option** 對話框直接建立）：Explicit Enhancement 專用的 Enhancement Spot，正確綁定到 **ZR_EN07_EXPLICIT_DEMO**。
2. **ZR_EN07_EXPLICIT_DEMO**（\$TMP，基礎程式由 ADT 建立，Explicit Enhancement 宣告與內容由使用者於 **SE38** 建立）：內含 **ENHANCEMENT-POINT** **ep_en07_after_init** **SPOTS** **zes_en07_v3**。

3. **ZEI_EN07_INSERT_DEMO** (使用者於 **SE38** 建立)：掛在上述插入點的 **Enhancement Implementation**，內容為修改 **lv_text** 並額外 **WRITE** 一行。已用 **programrun** 無頭執行驗證成功，輸出三行文字，證實插入的程式碼確實執行、且影響了插入點之後的既有邏輯。
4. **ZES_EN07_EXPLICIT_DEMO** / **ZES_EN07_POINT_DEMO**：本題排錯過程中建立的中間產物——2026-07-30 確認根因是用 **SE18** 建立的 **Spot** 型別是 **BAdI Definition**，天生就不適用於 **Explicit Point/Section**，留在系統裡當反面教材，未被實際使用。
5. **ZR_EN07_SECTION_DEMO** (**\$TMP**，使用者於 **SE38 Create Option** 對話框直接建立)：**ENHANCEMENT-SECTION es_mytest SPOTS zes_en07_section_v1**。包住一段預設金額計算邏輯，已端對端驗證 **Spot** 正確 **Binding** (**SE18 Technical Details** + 編輯器螺旋圖示雙重確認)。
6. **ZES_EN07_SECTION_V1** (隨上述 **Create Option** 動作自動建立並綁定，**Enhancement Method** 確認為 **Source Code Plug-In**)：**ZR_EN07_SECTION_DEMO** 的 **Explicit Enhancement Section** 專用 **Spot**。
7. **ZEI_EN07_SECTION_APPEND** (使用者於 **SE38** 建立，掛在 **ZES_EN07_SECTION_V1** 上)：已用 **programrun** 無頭執行驗證成功，確認 **ENHANCEMENT-SECTION** 在這套系統的 **Program** 型別情境下是整段取代，原邏輯 (含區域變數宣告) 完全不執行、對 **Implementation** 也不可見。

題目需求

1. 畫出 **Create Option** 與 **Create Implementation** 這兩個階段各自該用哪種編輯模式 (一般 **Change** 模式 vs **Enhance** 模式)、**Enhance** 按鈕該在哪個時間點按下，並解釋為什麼「先用 **SE18** 建立 **Spot**、再套用到 **Explicit Point/Section**」會得到「**nested enhancements**」這種聽起來像是物件設計錯誤、實際上是型別不合的誤導性訊息。
2. 解釋為什麼這支程式一旦有了 **Explicit Enhancement**，就不能再用 **ADT sap_lock** 編輯：這對「開發流程要不要優先用 **ADT** / **MCP** 自動化」這件事，帶來什麼實務上的提醒？
3. 對比 **ENHANCEMENT-POINT** 跟 **en04** 的 **Implicit Enhancement (Source Code Plugin)**：兩者都能「插入程式碼、不能改介面」，但一個要原開發者主動宣告、一個到處都有——如果你是原始程式的開發者，什麼情況下你會想主動加一個 **ENHANCEMENT-POINT**，而不是依賴到處都有的 **Implicit** 插入點？
4. 解釋 **ENHANCEMENT-SECTION** 的「整段取代」能力，跟 **en05/en06** 學到的哪個機制概念上最接近 (提示：想想 **Multi Use BAdI** 的多個 **Implementation** 依序疊加 **CHANGING** 參數 vs 這裡的「取代」，是相同的資料流向模式嗎？)。
5. 實測發現 **Implementation** 程式碼不能引用原邏輯裡宣告的區域變數 (**Field "LV_AMOUNT" is unknown**)，且執行結果證實「預設金額: 100」完全不會輸出——這兩個證據合起來，能不能直接推論出「這套系統的 **ENHANCEMENT-SECTION** 只有整段取代、沒有追加」這個結論？只看其中一個證據 (例如只看語法錯誤，不做實際執行測試) 夠不夠下這個結論？

參考答案

兩階段、兩種模式：① **Create Option** (第一次建立 **ENHANCEMENT-POINT/ENHANCEMENT-SECTION** 宣告 + 順便建立正確型別的 **Spot**) 全程用一般 **Change** 模式，不需要按 **Enhance**；完成後編輯器裝訂線會出現螺旋圖示，代表 **Binding** 成功。② 看到螺旋圖示之後才按 **Enhance** 切換按鈕，畫面標題變成「**Change Enhancements for ...**」，這時候游標點在 **SPOTS** 那一行、右鍵 **Enhancement Operations** 的 **Create Implementation** 才會是可用選項——因為系統要先確認這個位置已經有一個正確 **Binding** 的 **Spot**，才允許你掛 **Implementation** 上去。「**nested enhancements**」/「**spot defines enhancement spot in another object**」這類誤導性訊息，真正根因不是操作模式，是 **Create Option** 這一步用了 **SE18** 建立的 **Spot** (型別是 **BAdI Definition**) 去套用——**SE18** 從設計上就做不出 **Source Code Plug-In** 型別的 **Spot**，型別不合，系統只能用一堆語意不相關的訊息來表達「這個 **Spot** 用不了」。這是本題排錯過程中花費最多時間的地方：這些訊息表面上都指向「物件設定有問

題」，誘導人去反覆修改 Spot 名稱、程式碼結構，而不是去檢查「這個 Spot 到底是從哪裡建立的、型別對不對」這個更根本的前提。

ADT 鎖定限制的實務提醒： `ExceptionResourceIsEnhanced` 代表一旦某支 Z 程式被加上 Explicit Enhancement，未來所有對這支程式的修改（不管是不是跟 Enhancement 相關的部分）都必須回到 SAP GUI 進行，**ADT/MCP 自動化在這支程式上完全失效**。這提醒我們：如果一支自己寫的 Z 程式未來可能需要頻繁用 ADT / Claude 協助維護，要謹慎考慮是否要在它身上加 Explicit Enhancement Point——這類「主動開放給別人插入客製化」的設計，是用「未來這支程式對自動化工具關閉」換來的，是一個值得跟團隊溝通清楚的技術債 / 取捨決策，不是沒有代價的功能。

主動加 ENHANCEMENT-POINT vs 依賴 Implicit 的判斷：如果你明確知道「這個位置未來很可能需要讓客製化邏輯掛進來」（例如一段驗證邏輯、一段格式化邏輯），主動宣告 `ENHANCEMENT-POINT` 並取一個有語意的名稱（如 `ep_en07_after_init`），能讓未來要客製化的人一眼就看到這是官方預留的插入點，不用像 en04 那樣還要切換「Show Implicit Enhancement Options」才找得到，也不用擔心插入位置選得不好影響到不該影響的程式碼段落；Implicit 插入點雖然到處都有、彈性最大，但正因為到處都有，反而沒有「這裡才是建議插入點」的語意指引，客製化的人要自己判斷插入在哪裡最安全，風險相對更高（en04 就實際踩過這個風險：一開始用「不論任何條件、寫死測試值」的診斷版本測試，就真的建立了一張錯誤的正式工單）。

ENHANCEMENT-SECTION 取代能力跟 Multi Use BAdI 的資料流向對比：兩者不是同一種模式。Multi Use BAdI（en06）是「多個 Implementation 都執行、依序疊加同一個 `CHANGING` 參數」，原邏輯（Fallback 除外）不會被跳過；`ENHANCEMENT-SECTION` 的取代模式則是「Implementation 提供的程式碼完全取代原邏輯，原邏輯根本不會執行」，更接近 Single Use BAdI 的精神（保證只有一個邏輯真正生效，不會疊加）——`ENHANCEMENT-SECTION` 本身雖然也可以有多個 Implementation，但語意上是「挑一個取代」而不是「疊加處理」，跟 Multi Use BAdI「刻意讓多個邏輯依序處理同一份資料」的設計目的並不相同。

兩個證據合起來才夠下結論，只看語法錯誤不夠：`Field "LV_AMOUNT" is unknown` 這個語法錯誤只能證明「Implementation 的編譯期作用域看不到原邏輯的區域宣告」，理論上這也可能只是「取代」跟「追加」共通的一個技術限制（例如即使是追加模式，Implementation 程式碼也可能被編譯成獨立單元、本來就不共用原邏輯的區域變數），不能單靠這個語法錯誤就排除「追加」的可能性——追加也可能是「兩段各自獨立編譯，但執行時依序都跑」。真正一錘定音的是執行輸出：`WRITE '預設金額:', lv_amount.` 這一整行完全沒有出現在輸出裡，代表原邏輯在執行期根本沒有被觸發，不是「跑了但看不到變數」而是「整段沒跑」——這才是能排除「追加」、坐實「整段取代」的關鍵證據。這個對照本身是很好的方法論提醒：編譯期的錯誤訊息只能告訴你「靜態結構上發生了什麼」，要驗證「執行期實際發生了什麼」，一定要真的跑一次看輸出，兩者不能互相取代。

思考題

1. 這題排錯過程中，第一輪把「型別不合」的問題誤判成「操作模式不對（忘記按 Enhance）」，繞了不少路才在下一輪用真實 GUI 操作 + 截圖證據推翻。如果之後要幫團隊寫這門課的操作手冊，你會怎麼設計內容結構，讓「先講結論、附上截圖佐證」比「按時間順序講排錯過程」更能避免下一個學員重蹈覆轍？
2. 本題發現「SE18 建立的 Enhancement Spot 只能是 BAdI Definition 型別」，跟 en05「Enhancement Spot 建立本身就是 GUI-only」的教訓放在一起看，你會怎麼歸納「同一個 ADT 物件類型（`ENHS/XS`），實際可以建立出來的『子類型』，可能因為建立入口（SE18 vs SE38 Create Option）不同而不同」這個現象？如果之後遇到其他物件類型也有類似「畫面選項看起來很直覺，但其實漏了某個子類型」的狀況，你會怎麼提早發現，而不是等到套用失敗才知道？

3. 如果團隊裡有人堅持要用 ADT / Claude 全自動完成 Explicit Enhancement 的建立與內容維護 (不想碰 SAP GUI) , 你會怎麼跟他解釋這在目前這套系統是做不到的? 除了「ADT 不支援」這個事實陳述, 你還能提出什麼替代建議, 讓他的維護流程負擔降到最低? (提示: 例如「把會插入的程式碼邏輯集中寫在一個獨立的 Z Class 方法裡, Enhancement 裡只放一行呼叫這個方法」——這樣真正需要在 SAP GUI 手動維護的程式碼量降到最低, 大部分邏輯還是能用 ADT 自由編輯)